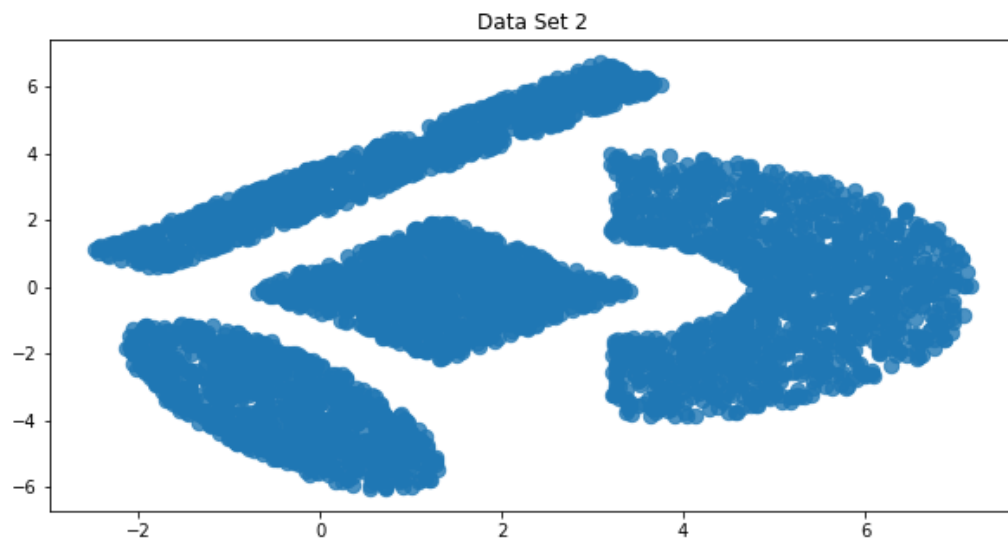
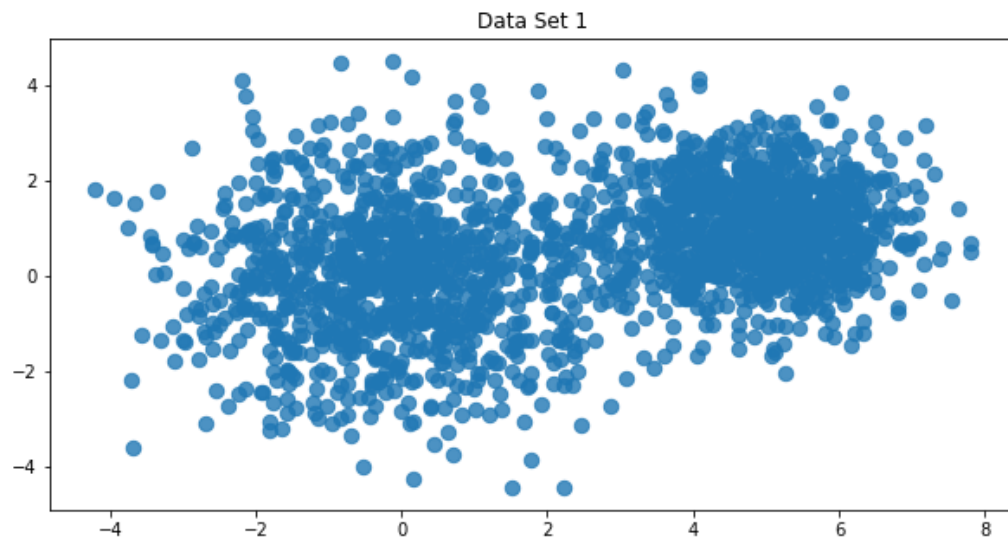


IE529 – Computational Assignment 2
Jigar Patel (jbpatel4)

1. Include a scatter plot for each set of data, as is, i.e., with no clusters identified yet



2. Submit your pseudocodes for the Lloyds/K-means algorithm, the GreedyKcenters and Single Swap algorithms, and the Spectral Clustering algorithm, each on a separate sheet of paper. State clearly whether you are considering normalized or unnormalized Spectral Clustering

A. Lloyd's/K-means Algorithm

1. Initialization: arbitrarily select k centers, pick tolerance
2. Assign each x_i to its nearest center m_j (in squared 2-norm)
3. Update: (i) recompute new position of each center based on which points x_i are in its cluster, (ii) error: 2-norm based as given in assignment
4. Repeat 2,3 until tolerance

B. Greedy K-Centers

1. Initialization: arbitrarily select 1 center
2. Pick farthest point from "Initialization" step and add it to list of centers
3. Compute distances from each point x_i to each center, assign closest center
4. Pick farthest point among those found in Step 3, and add to list of centers
5. Repeat 3,4 for k centers

C. Single-Swap

1. Set $C =$ size k subset of X (data)
2. While there exists $m_j \in C$ and $x_i \in X \setminus C$
Such that:
$$\text{Cost}(C \setminus m_j \cup x_i) < \text{Cost}(C)$$
3. Set $C = C \setminus m_j \cup x_i$
4. End While

D. Spectral Clustering (non-normalized)

1. Start with weighting/similarity matrix W (assuming W is provided)
2. Construct Laplacian L
3. Compute eigenvalues and eigenvectors of L
4. Consider only first k eigenvectors (those associated with k smallest eigenvalues)
5. Construct Z from taking the 'rows' of eigenvectors from Step 4

6. Apply K-means on Z

3. For your K-means algorithm by itself, and for your Spectral Clustering algorithm, present the output (clustering) results for BOTH sets of test data given to you (see below).
4. Evaluate the results for a range of K values (number of clusters) from $K = 2$, to $K = 11$ (but not all of these; try 3 different values, then try to pick 2-3 more that will help you find the best cluster outcome). When implementing the K-means algorithm (by itself and from your Spectral Clustering algorithm), you should try multiple initializations (say 5 to 10) and then select the 'best' result, for each K value. State how many different initializations you tried and why.
 - The output should consist of a scatter plot of the data with the clusters differentiated by color, and with the centroids highlighted in a different color, size and shape than the corresponding cluster members.

The tests I ran are summarized in the table below.

Test	Algorithm	Data	K	Tolerance
1	Lloyd	1	$K = 2, 5, 8$	$1e-5$
2	Lloyd	2	$K = 2, 5, 8$	$1e-5$
3	Lloyd	1	$K = 2, 5, 8$	$1e-2^*$
4	Lloyd	2	$K = 2, 5, 8$	$1e-2^*$
5	Spectral	1	$K = 2, 5, 8$	$1e-5$
6	Spectral	6	$K = 2, 5, 8$	$1e-5$

Tests for Model Validation

- *Note 1: I chose to lower the tolerance as I noticed there was no big change in centroid location after error goes below $1e-4$
- Note 2: I tried 6 different initializations for each cluster size for each test. For the conciseness of the report, I'm presenting my 'best' result.

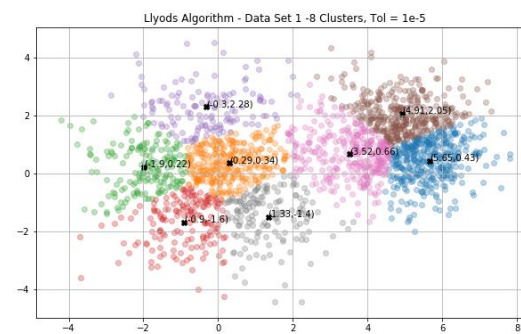
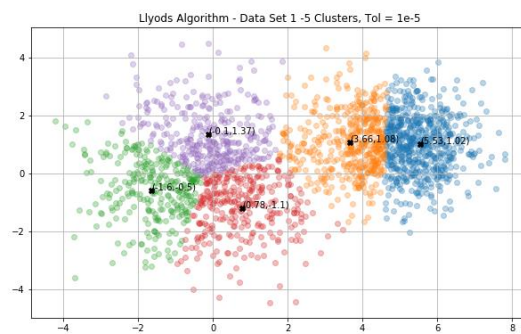
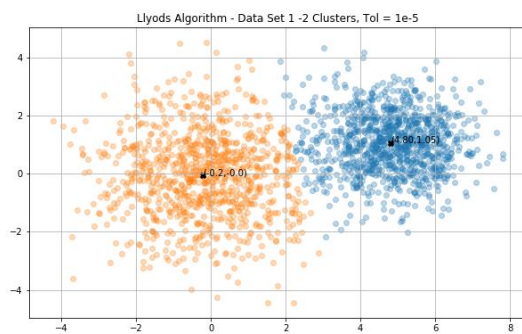
We first focus on Llyod's Algorithm (Tests 1 to 4).

The assignment vector is generated for each test and trial. For the conciseness of the report, I'm including a snapshot for one of the test to give an idea what it looks like.

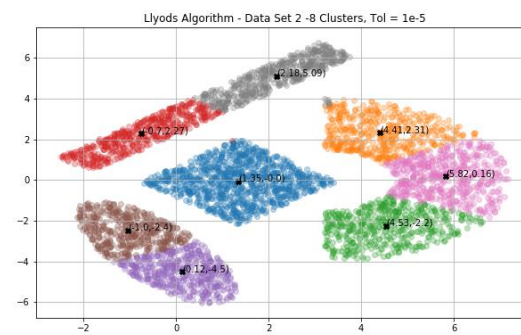
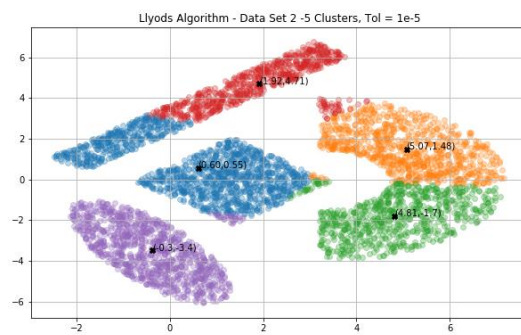
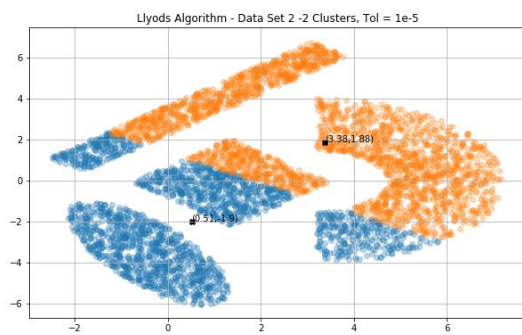
	x	y	1	2	3	4	5	6	7	8	assign_center
0	0.179780	-1.034300	5.656937	1.388538	2.506437	1.287011	3.356034	5.679085	3.756996	1.247652	8
1	-0.794470	-0.367580	6.490879	1.301513	1.332328	1.334459	2.689988	6.233563	4.438743	2.413512	2
2	-1.392000	-0.026950	7.055385	1.725157	0.647197	1.733070	2.540864	6.675686	4.960065	3.101055	3
3	-0.652190	-0.749510	6.407453	1.448774	1.652366	0.984543	3.048753	6.262902	4.410622	2.126365	4
4	-1.225900	1.765400	7.008827	2.075261	1.724070	3.474229	1.032319	6.188386	4.860587	4.149655	5
5	-0.027022	0.608130	5.680612	0.410176	2.001201	2.472396	1.701911	5.184416	3.543369	2.509266	2
6	0.010661	0.208670	5.642853	0.314303	2.000919	2.121502	2.101935	5.273081	3.538168	2.161618	2
7	-0.016534	-1.768900	6.069962	2.140914	2.802005	0.910684	4.063616	6.262466	4.306162	1.381951	4
8	1.274800	1.499600	4.509591	1.512948	3.506813	3.878909	1.787695	3.720769	2.380689	2.997371	2
9	0.353740	-0.259620	5.338074	0.612521	2.392535	1.922414	2.633011	5.144067	3.302438	1.581439	2
10	0.392980	1.133000	5.306942	0.789809	2.552100	3.120379	1.359164	4.652268	3.152925	2.794865	2
11	0.761970	-0.688650	5.009346	1.139980	2.898318	1.964074	3.166210	5.002508	3.081296	0.992917	8
12	1.204700	2.474700	4.902688	2.312869	3.910392	4.682398	1.549064	3.777217	2.916723	3.974046	5
13	1.869700	1.384600	3.904689	1.887212	4.031785	4.158678	2.378079	3.155065	1.784362	2.930260	7
14	-0.744900	-2.897100	7.198065	3.408242	3.356669	1.214705	5.195924	7.540198	5.572265	2.510030	4
15	0.236380	1.268500	5.481768	0.920420	2.461118	3.183342	1.162554	4.782332	3.328740	2.977083	2
16	-0.365020	1.150000	6.060764	1.035126	1.872477	2.900144	1.132893	5.394999	3.906931	3.147830	2
17	1.200900	-2.890500	5.537888	3.365422	4.456296	2.438007	5.395377	6.195808	4.267313	1.400174	8
18	-0.610250	-0.171720	6.286970	1.041692	1.434499	1.556008	2.469823	5.989046	4.215535	2.356682	2
19	1.034800	0.300470	4.616457	0.744900	3.026096	2.796910	2.407764	4.289109	2.511620	1.823028	2

Cluster Assignment for Llyod's Algorithm (K-Means)

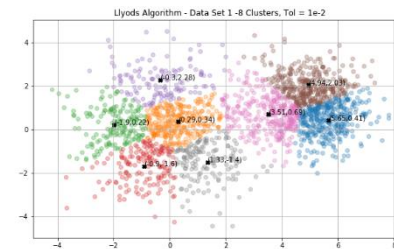
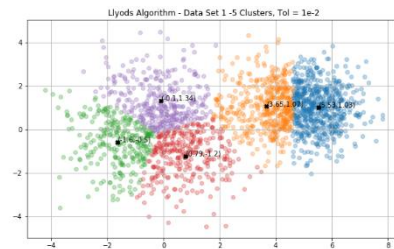
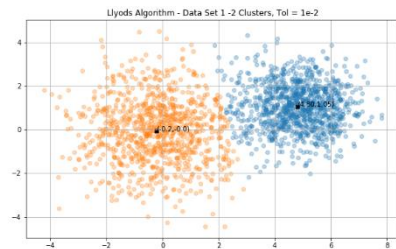
Test 1



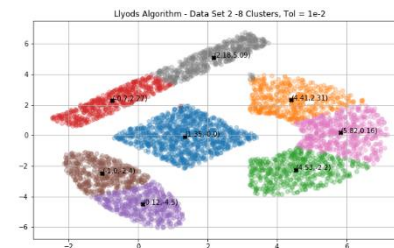
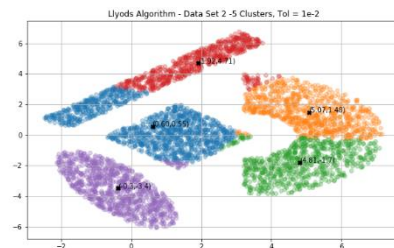
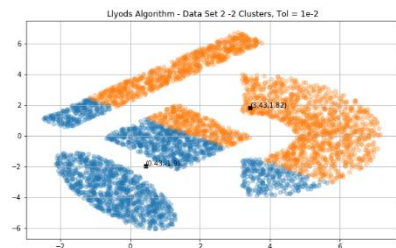
Test 2



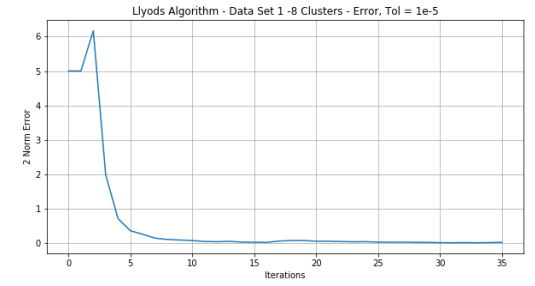
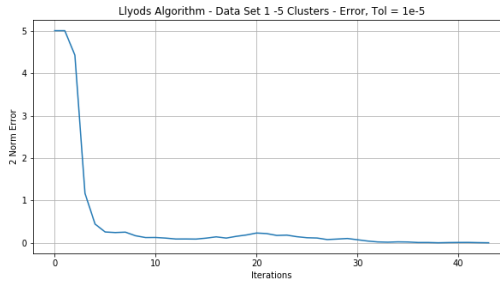
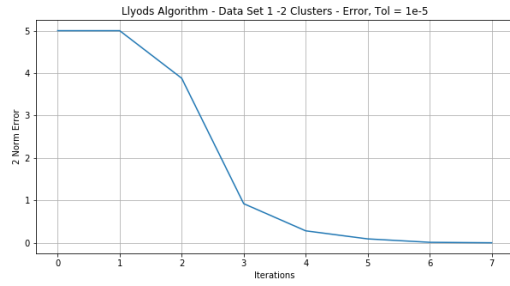
Test 3



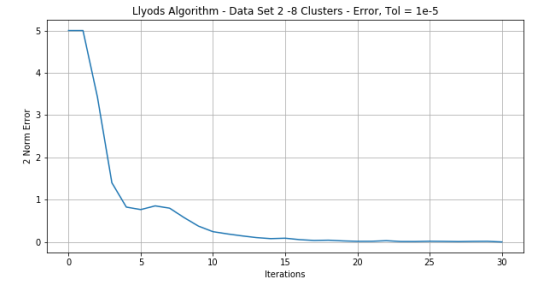
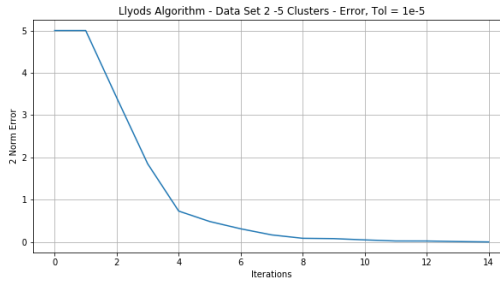
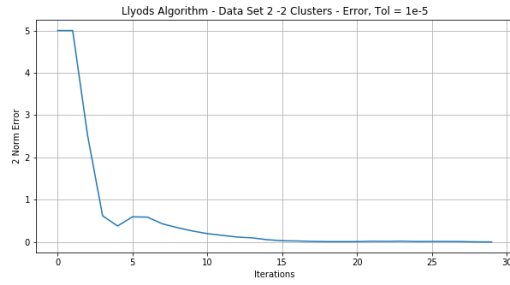
Test 4



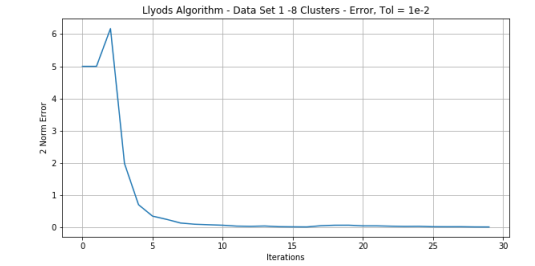
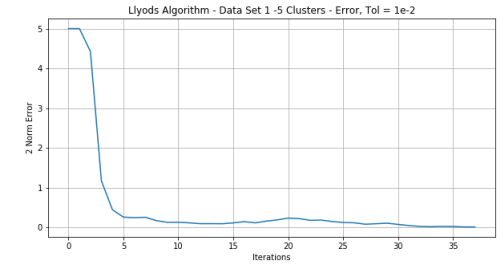
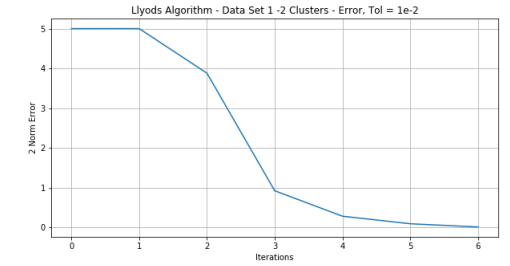
Test 1



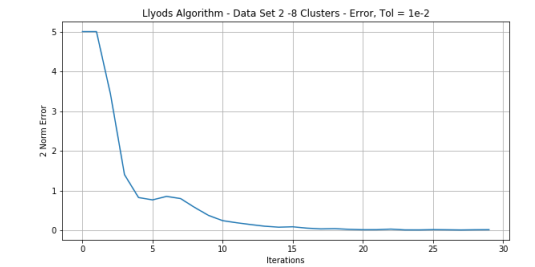
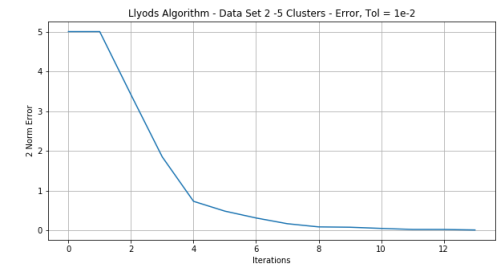
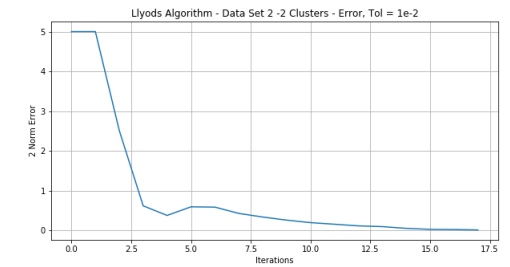
Test 2



Test 3



Test 4



Now, we focus on Spectral Clustering (Tests 5 and 6).

The similarity matrix is constructed. For the conciseness of the report, I'm including a snapshot for one of the Data Set 1 to give an idea what it looks like. The similarity matrix is constructed using the Gaussian distance with $\sigma = 2$ (based on online literature research)

	0	1	2	3	4	5	6	7	8	9	...	1990
0	1.000000	0.554177	0.393198	0.644242	0.208798	0.437054	0.534079	0.683732	0.251530	0.672344	...	0.157406
1	0.554177	1.000000	0.708999	0.815637	0.336860	0.537577	0.609540	0.448706	0.248186	0.561784	...	0.112748
2	0.393198	0.708999	1.000000	0.596270	0.406564	0.471072	0.491076	0.329637	0.215153	0.414539	...	0.087080
3	0.644242	0.815637	0.596270	1.000000	0.275337	0.473628	0.558470	0.548444	0.227442	0.571530	...	0.114053
4	0.208798	0.336860	0.406564	0.275337	1.000000	0.434676	0.370077	0.154472	0.284394	0.276891	...	0.092016

Similarity Matrix – Data Set 1

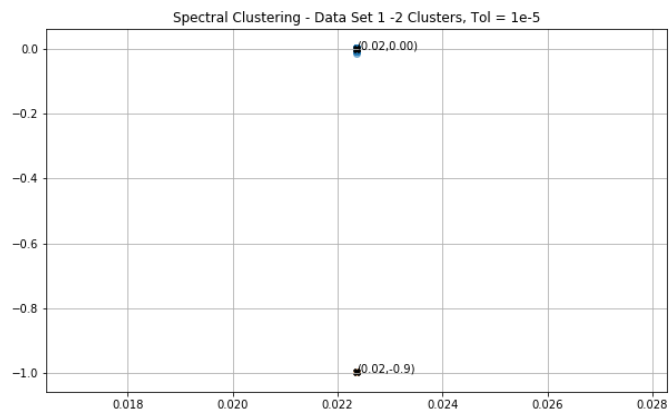
The eigen decomposition of the Laplacian matrix is then computed. The first four eigenvalues are shown along with the two of the eigenvectors corresponding.

```
Four Smallest Eigenvalues:
[ 1.77562449e-14  9.81658187e+01  1.25674595e+02  1.32370508e+02]

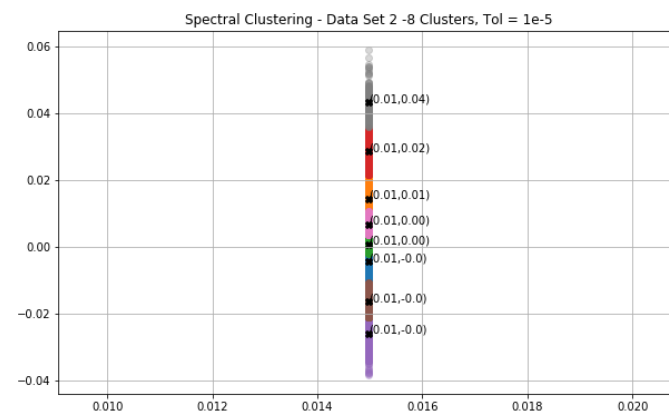
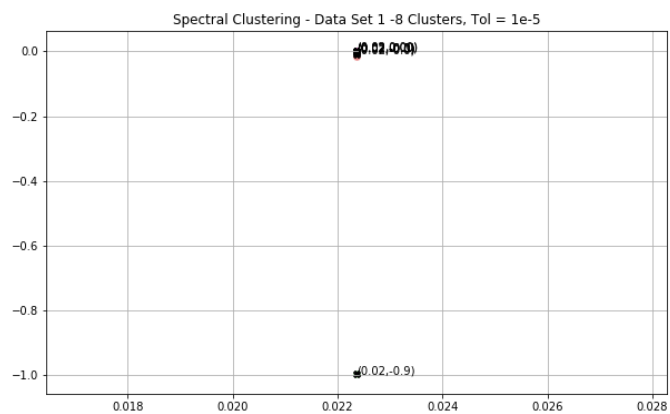
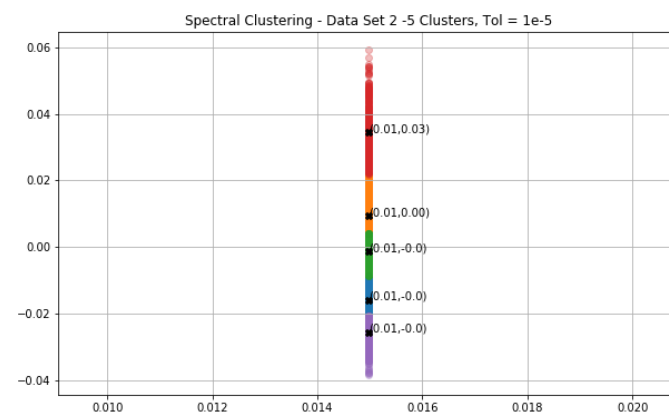
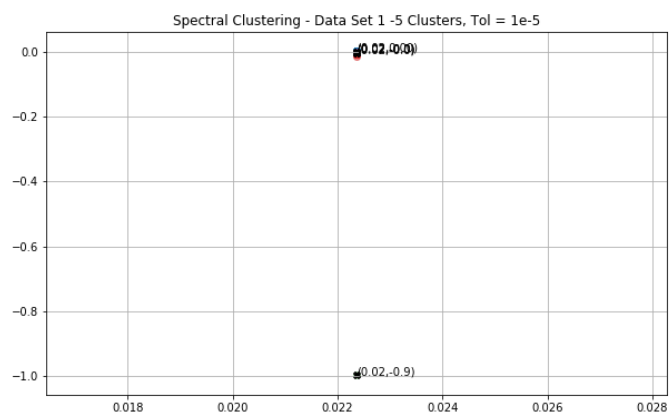
First two eigenvectors:
[[ 0.02236068 -0.00068451]
 [ 0.02236068 -0.00103254]
 [ 0.02236068 -0.00127938]
 [ 0.02236068 -0.00107466]
 [ 0.02236068 -0.00078633]
 [ 0.02236068 -0.00041978]
 [ 0.02236068 -0.00047872]
 [ 0.02236068 -0.00106091]
 [ 0.02236068  0.00037643]
 [ 0.02236068 -0.00040546]]
```

Eigenvalues and Eigenvectors of Laplacian

Test 5

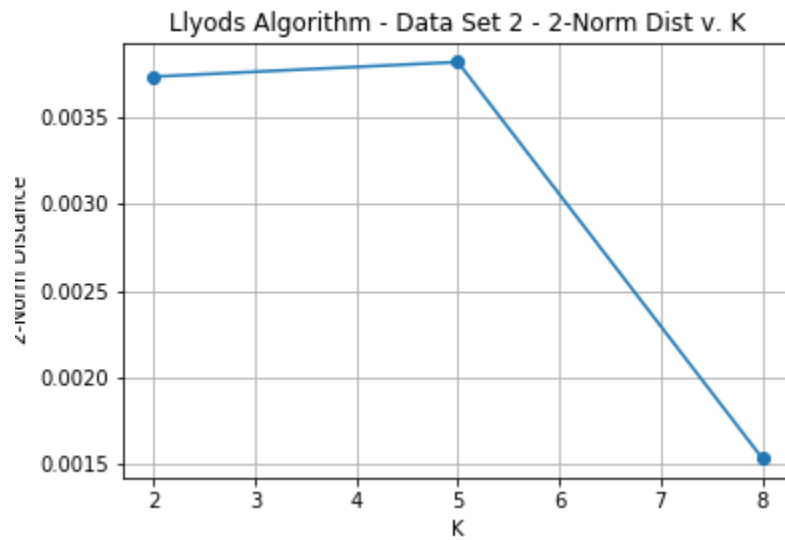
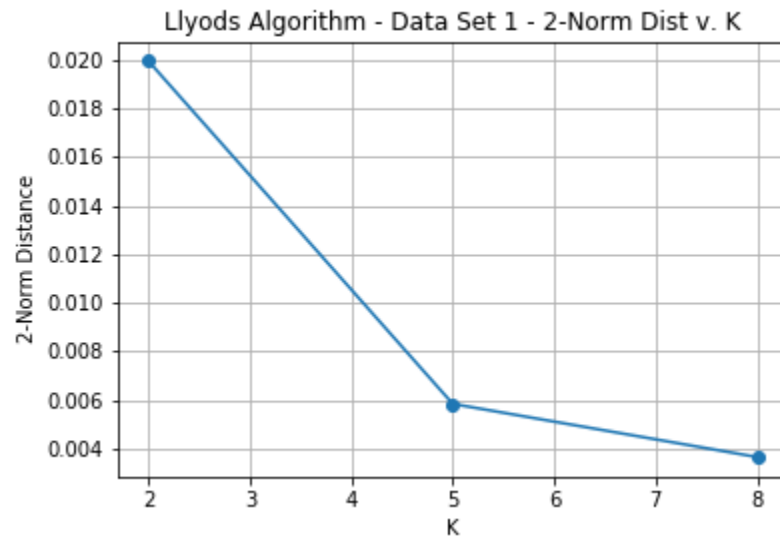


Test 6



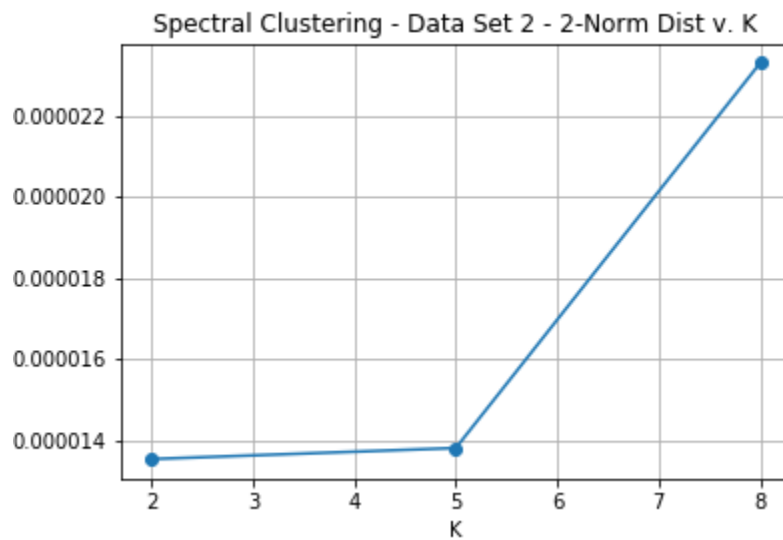
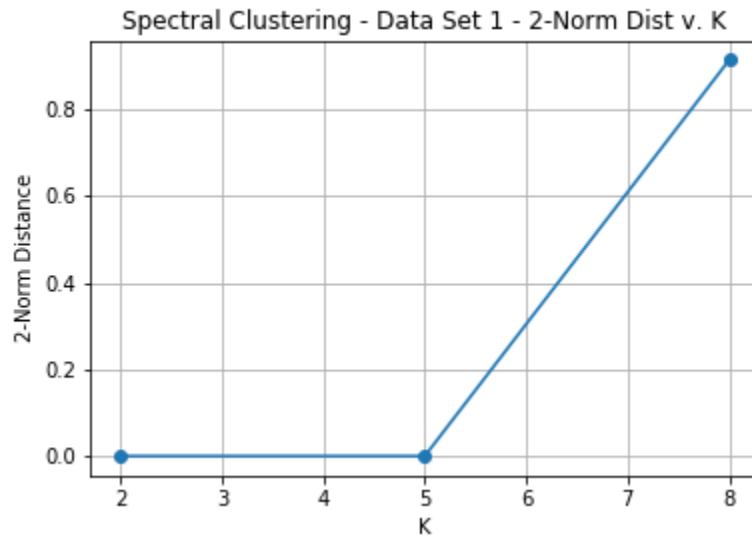
- A plot of the distance metric, D , versus, K , for the best outcomes found for each of the K values you considered, for both K-means and Spectral approaches

For Lloyd's Algorithm:



We can see that generally, as you increase the number of clusters, you will have better convergence to the solution for K-means algorithm.

For Spectral Clustering:

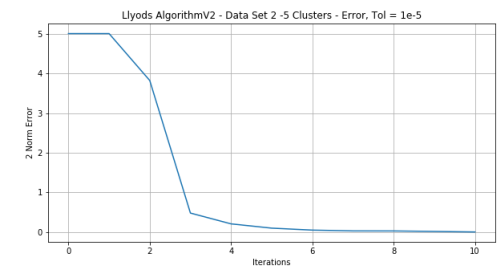
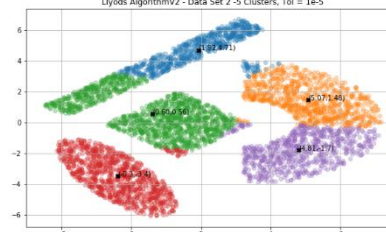
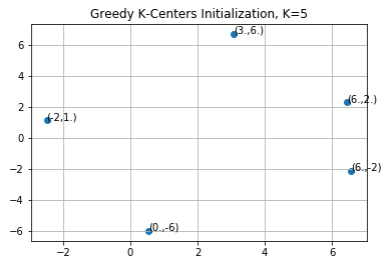


We can see that generally, as you increase the number of clusters, you will have divergence from the solution with Spectral Clustering.

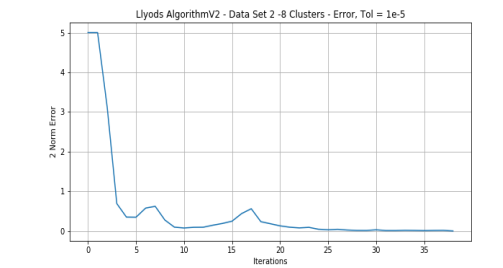
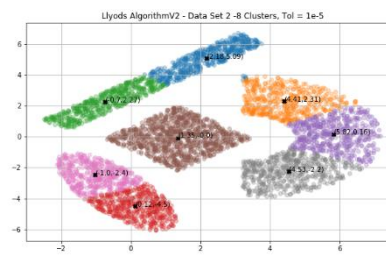
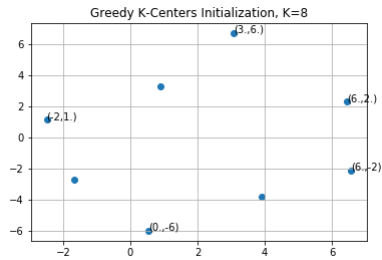
- Using the two best results from your K-means, that is, two different K-values that seem to give the best clustering result, use your GreedyKcenters algorithm to find an initialization and re-run K-means with this initialization. Describe how this does or does not improve your results.

From K-means (Test 1-4) we can see that K=5 and K=8 seem to give the best clustering result.

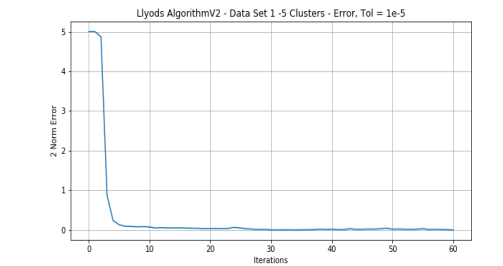
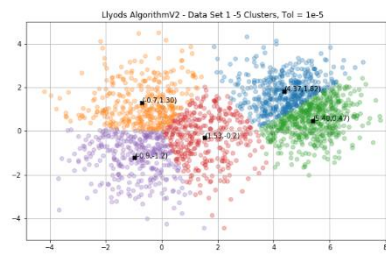
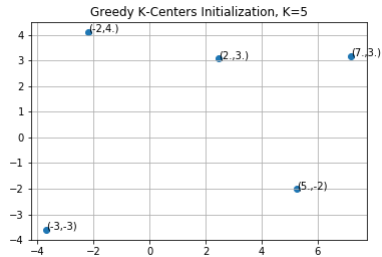
Data Set 2, K=5



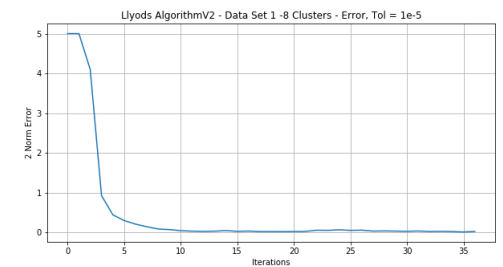
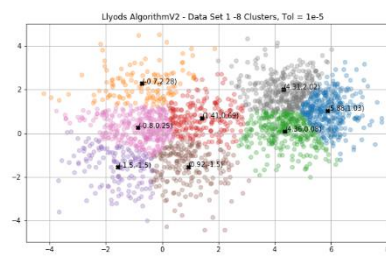
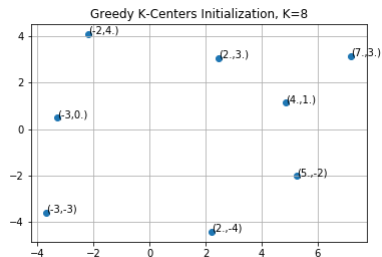
Data Set 2, K=8



Data Set 1, K=5



Data Set 1, K=8



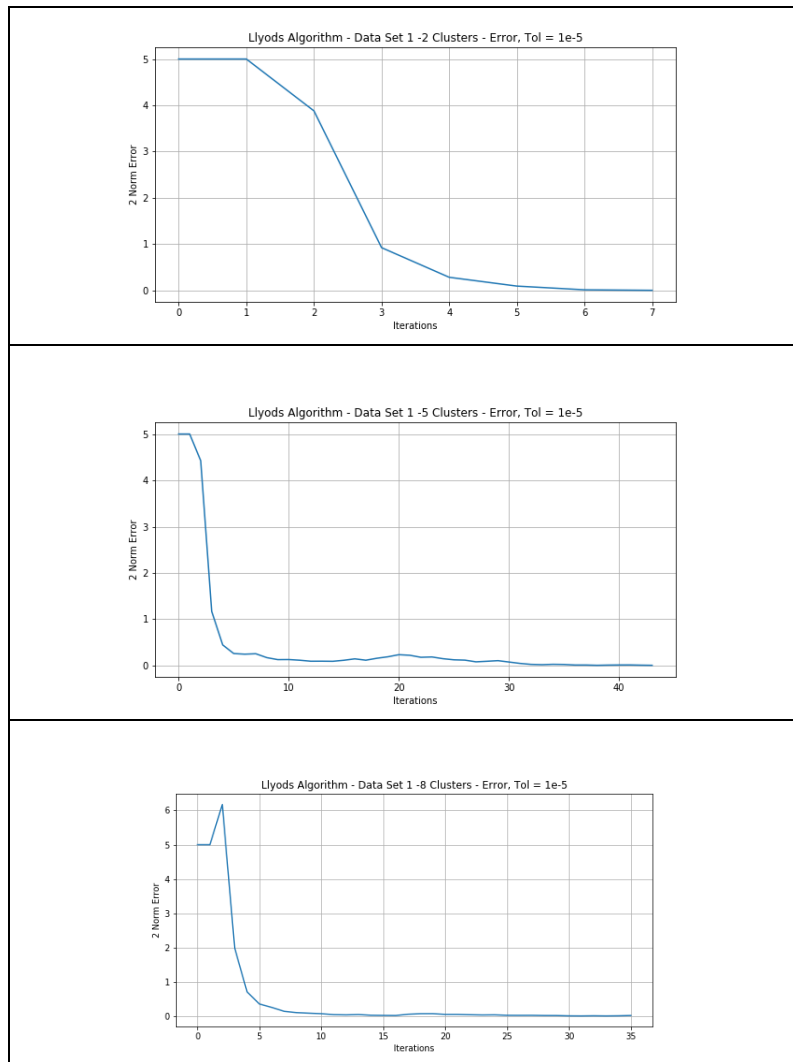
We can observe that the Greedy K-Centers initializations helps converge to the solution faster, as evident by the smaller number of iterations.

5. For the K-means algorithm, describe how you tested for convergence and why you choose this as a test

The convergence condition used was the 2-norm distance measure as suggested in the assignment prompt. For each of the test, I computed the 'error' and once it fell below the given threshold, the algorithm terminates. If we let Y represent the solution at iteration p , then we can compute the 'error' as

$$\|Y_p - Y_{p+1}\|_2 < tol$$

This convergence test is used as it measures the difference between two set of points, in our case the set of centers. If we notice that the newly obtained centers haven't moved much (above the tolerance), then we can assume we are at the solution. Again, for the conciseness of the report, I've included the error charts for Test 1. A summary is shown in the previous part where the distance metric is plotted against different K .



6. Compare the results you get using your spectral clustering algorithm to the results obtained with your K-means algorithms for each of the data sets; you should compare the objective function values (D) and discuss qualitatively how well you think each clustering method worked (i.e., base this comparison on the results given in 4).

Refer to Tests 1-4 and Test 1-6 for results comparing K-means and Spectral Clustering, respectively. In summary, there is no clear pattern on whether the objective function values D are better for one than another. However, we can say that spectral clustering is better at uncovering clusters from 'highly mixed' data. We can see that the error stays almost zero for Data Set 1 which is more clustered than Data Set 2, which has an increase in error. The advantage for Spectral Clustering is relatively low computational complexity for small number of components. K-means is not entirely reliable as the initial choice of centers greatly influences how long, if at all, we will converge to a solution. However, we saw an improvement in its performance when we used Greedy K-Centers to choose an initial set of centers.

7. Discuss how many "natural" clusters you think each data set has, and why.

Data set 1 appears to have 2 natural clusters. We can see roughly two blobs formed by the data, however it is difficult to tell. Throughout the analysis, more than 2 clusters also seemed very plausibly. The reason for such clustering might be that what we are measuring has low intrinsic dimensionality and thus we need to apply clustering techniques to recover the main components.

Data set 2 has 4 natural clusters. This is easy to deduce as the clusters can be clearly seen as different geometric shapes. The reason for such clustering may have to do with the fact that what we are measuring naturally falls into different clusters. So at this point, there is not too much use for applying clustering techniques as visual inspection can suffice.

8. Provide a short analysis of the computational effort you found was required for each of the algorithms. State what measure you used to evaluate computational effort (e.g., running time or flops, or ??), and how you observed this. Briefly discuss how these computational efforts compare to your expectations.

Algorithm	Computational Cost	Observe	Expectations
-----------	--------------------	---------	--------------

K-Means	$O(nkd)$	1.2s	-knew it would take some time as there is no guarantee from initial set of clusters
Greedy K-Centers	$O(nkd)$	2s	-computing distance repeatedly would slow down convergence
Swap	$(n C k)$ worst case	>3 min	-more complex than K-means as now you are running K-means multiple times
Spectral		1 min	-computing eigenvectors and eigenvalues of large dimensional matrix very expensive

9. Write a brief summary paragraph of your findings, that is, briefly summarize your KEY findings.

The comparison of the four algorithms revealed computational challenges as well as mathematical guarantees for an acceptable solution. In K-means it was found that it is both computationally expensive and doesn't mathematically guarantee an optimal solution. We simply terminate when the centers don't move, but that doesn't mean it is the optimal solution. The Greedy K-Centers is equally worse and found to be more computationally expensive due to multiple distance calculations. Further the Swap is even worse as you are running K-means multiple times. Although it has as mathematical convergence guarantee, it is inefficient for wide practical use as seen by its high running time. Finally, Spectral Clustering shows some promise in its accuracy to recover clusters from data with low intrinsic dimensionality, however it is also very numerically unstable due to calculation of eigenvectors of high dimensional matrices.

10. Appendix

K-Means

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def initial_cluster(data, endk):

    init_clust = np.zeros((endk,2))

    for i in range(startk,endk):
        init_clust[i,0] = data.iloc[i]['x']
        init_clust[i,1] = data.iloc[i]['y']

    #print("\n Initial Centers: \n", init_clust, "\n")

    return init_clust

def assign_cluster(data, cluster):

    for i in np.arange(0,cluster.shape[0]):

        data['{}'.format(i+1)] = np.sqrt(
            (data['x'] - cluster[i,0])**2 +
            (data['y'] - cluster[i,1])**2 )

        data['center'] = data[list(map(str,
list(range(1,cluster.shape[0]+1))))].idxmin(axis=1)

    return data

def update(data, centers):

    #print("\n Old Centers: \n", centers)
    old = np.array(centers)

    new_x = np.array(data.groupby('center')['x'].mean()).T
    new_y = np.array(data.groupby('center')['y'].mean()).T

    centers[:,0] = new_x
    centers[:,1] = new_y

    newc = np.array(centers)
```

```

plt.scatter(new_x, new_y)
#print("\n Updated Centers: \n" ,centers, "\n")

error = np.linalg.norm((newc-old),2)

return [centers, error]

def graph_out(df,clust_dict,endk):
    plt.figure(figsize=(10,6))
    plt.grid(True)
    fck = df.groupby('center')

    for key, item in fck:
        plt.scatter( (fck.get_group(key))['x'] , (fck.get_group(key))['y'], alpha=0.3 )

    for i in range(0,endk):
        plt.scatter( clust_dict[i,0], clust_dict[i,1], marker = 'X',c= 'k' )
        str1 = str(clust_dict[i,0])[:4]
        str2 = str(clust_dict[i,1])[:4]
        str3 = "(" + str1 + "," + str2 + ")"
        plt.annotate((str3), (clust_dict[i,0], clust_dict[i,1]), fontsize=10)

    plt.title('Llyods Algorithm - Data Set 1 -' + str(endk) + ' Clusters, Tol = 1e-5')
    plt.gcf()
    plt.savefig('Llyods Algorithm - Data Set 1 - {} Clusters, Tol = 1e-5'.format(endk))

def k_means(data, endk):
    clust_dict = {}
    iteration = 0
    error = 5
    elist = []
    elist.append(error)

    # 1. Initialize clusters
    clust_dict[iteration] = initial_cluster(data, endk)

    plt.figure(figsize=(10,5))
    plt.grid(True)

    while (error>1e-5):

```

```

elist.append(error)

# Assign each (x,y) to a cluster
assign_cluster(data,clust_dict[iteration])

# Update each cluster
upt = update(data, clust_dict[iteration])
clust_dict[iteration+1] = upt[0]
error = upt[1]

iteration = iteration + 1

#plot data and clusters

#plt.scatter( clust_dict[iteration][:,0], clust_dict[iteration][:,1],s=70,alpha=0.8)
#plt.legend([i for i in range (1,iteration, 5)])
#plt.scatter(data.iloc[:,0], data.iloc[:,1],alpha=0.20,s=10)
#plt.title('Llyods Algorithm - Data Set 1 -' + str(endk) + ' Clusters, Tol = 1e-2')
#plt.gcf()
#plt.savefig('Hello123') #'Llyods Algorithm - Data Set 1 - {} Clusters, Tol = 1e-
2'.format(endk)

#plot error
plt.figure(figsize = (10,5))
plt.grid(True)
plt.plot(np.array(elist))
plt.title('Llyods Algorithm - Data Set 1 -' + str(endk) + ' Clusters - Error, Tol = 1e-
5')
plt.ylabel('2 Norm Error')
plt.xlabel('Iterations')
plt.gcf()
plt.savefig('Llyods Algorithm Error - Data Set 1 - {} Clusters, Tol = 1e-
5'.format(endk)) #

#plt.show()

graph_out(df, clust_dict[iteration],endk)

return [clust_dict, np.array(elist)]

```

```

#read in data
df = pd.read_csv('data2.csv',header = None, names = ['x','y'])
erd = []
ktrk = []

for endk in range(2,11,3):
    #solve it
    sol = k_means(df, endk)
    ktrk.append(endk)
    erd.append(sol[1][-1])

plt.figure()
plt.grid(True)
plt.plot(np.flip(ktrk,0),erd,marker='o')
plt.title('Llyods Algorithm - Data Set 2 - 2-Norm Dist v. K')
plt.xlabel('K')
plt.ylabel('2-Norm Distance')
plt.gcf()
plt.savefig('Llyods Algorithm - Data Set 2 - 2-Norm Dist v. K.png')
plt.show()

```

Greedy K-Centers

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def initialize(data):

    maxidx = [ data[0,0], data[0,1] ]
    d = np.sqrt( (data[:,0]-maxidx[0])**2 + (data[:,1]-maxidx[1])**2 )
    d = d.reshape((data.shape[0],1))
    data = np.append(data, d, axis = 1)

    return data

def get_max(data):
    maxidx = data[(data[:,-1]).argmax(),:2]
    return maxidx

def distance(data, idx):

    distance = np.sqrt( (data[:,0]-idx[0])**2 + (data[:,1]-idx[1])**2 )
    distance = distance.reshape((data.shape[0],1))

```

```

data = np.append(data,distance,axis=1)

return data

def get_min(data):

    mincol = np.empty((data.shape[0],1))

    for i in range(0, data.shape[0]):
        mincol[i] = min(data[i,2:])

    data = np.append(data, mincol, axis=1)

    #minidx = data[(data[:,-1]).argmin(),:2]
    maxidx = get_max(data)

    data = data[:,-1]
    #print(data)

    return maxidx

dframe = pd.read_csv('data1.csv',header = None, names = ['x','y'])
dary = dframe.as_matrix()
k=8
a = initialize(dary)
q = np.zeros((k,3))
itr = 0
q[itr,:2] = a[(a[:,-1]).argmax(),:2]

for iteration in range(1,k):
    a = distance(a,q[itr])
    q[itr,2] = np.sum(a[:,-1], axis=0)
    itr = itr+1
    q[itr,:2] = get_min(a)

greedy_df = pd.DataFrame(q,columns=['x','y','2-Norm Dist'])

plt.figure()
plt.grid(True)

for i in range(0,8):

    str1 = str(greedy_df.iloc[i,0]):2]
    str2 = str(greedy_df.iloc[i,1]):2]
    str3 = "(" + str1 + "," + str2 + ")"

```

```

plt.annotate((str3), (greedy_df.iloc[i,0], greedy_df.iloc[i,1]), fontsize=10)

plt.scatter(greedy_df['x'], greedy_df['y'])
plt.title('Greedy K-Centers Initialization, K=8')
plt.gcf()
plt.savefig('Greedy K-Centers Initialization, K=8')

```

Swap

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def swap_initialization(df, swapidx):
    swap_clust = initial_cluster(df, endk)
    swap_clust[-1,0] = df.iloc[swapidx]['x']
    swap_clust[-1,1] = df.iloc[swapidx]['y']

    return swap_clust

def calc_cost(df):
    cost = np.trace((df.groupby('center').sum()).as_matrix(map(str, [i for i in
range(1,5)])))
    return cost

def k_means_swap(data, swapidx):
    swap_clust_dict = {}
    iteration = 0
    error = 5
    elist = []
    elist.append(error)

    plt.figure()
    plt.grid(True)

    # 1. Initialize clusters
    swap_clust_dict[iteration] = swap_initialization(data, swapidx)

    while (error > 1e-5):

        elist.append(error)

        # Assign each (x,y) to a cluster

```

```

assign_cluster(data,swap_clust_dict[iteration])

# Update each cluster
upt = update(data, swap_clust_dict[iteration])
swap_clust_dict[iteration+1] = upt[0]
error = upt[1]

iteration = iteration + 1

#plot data and clusters
plt.legend([i for i in range (1,iteration)])
plt.scatter(data.iloc[:,0], data.iloc[:,1],alpha=0.15,s=1)

#plot error
plt.figure()
plt.plot(np.array(elist))
plt.xlabel('Iterations')

plt.show()

return swap_clust_dict

```

Spectral Clustering

```

def similarity(data)
    b = np.zeros((a.shape[0],a.shape[0]))

    for i in range(0,a.shape[0]):
        for j in range(0,a.shape[0]):
            b[i,j] = math.exp(-np.linalg.norm((a[i] - a[j]),2)/2)

def diagonal(b)
    d = np.zeros((a.shape[0],))

    for i in range(0,a.shape[0]):
        d[i] = np.sum(b[i],axis=0)

    print(d)
    D = np.zeros((a.shape[0],a.shape[0]))
    D = np.diag(np.array(d))

Def Laplacian(D,W)
    L = D-W
    val, vec = np.linalg.eig(l)

```

